

NumPy Fundamentals | Data Science Mentorship Program

1. Introduction & Theoretical Discussion

NumPy is an essential library for Data Analysts and Data Scientists. Next to Pandas, it is the fundamental building block of Data Science in Python. Most major machine learning and data science libraries (like Pandas, Matplotlib, Scikit-Learn, and TensorFlow) are built on top of NumPy.

What is NumPy?

NumPy (Numerical Python) is the fundamental package for scientific computing in Python.

- It provides a multidimensional array object called the **NumPy Array** (or ndarray).
- It contains derived objects like masked arrays and matrices.
- It includes a wide assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, discrete Fourier transforms, basic linear algebra, statistical operations, and random simulations.

Why do we need NumPy when Python has Lists?

In the early days of Data Science, languages like MATLAB and R were preferred because Python's built-in data structures (like lists and dictionaries) were considered too slow to handle gigabytes of data. Python lists are "referential arrays" holding mixed data types, which makes processing inherently slow.

However, Python's syntax was widely loved for its simplicity. To solve the speed issue, developers created the NumPy library:

- **Written in C:** The core of NumPy is written in C, providing execution speeds similar to C.
- **Operates in Python:** A wrapper was built around it so it could be easily used with Python's simple syntax.
- **Best of Both Worlds:** Speed of C + Comfort/Syntax of Python.

Differences between NumPy Arrays and Python Sequences (Lists)

1. **Fixed Size:** Unlike Python lists which can grow dynamically, NumPy arrays have a fixed size upon creation. Changing the size of an ndarray will create a new array and delete the original.
2. **Homogeneous Data Types:** Python lists can store mixed data types (integers, strings, booleans together). NumPy arrays require all elements to be of the **same data type**. Because of this, elements occupy the same size in memory, removing the need for referential pointers and making computations lightning fast.
3. **Advanced Mathematical Capabilities:** NumPy makes applying complex mathematical

operations on large datasets highly efficient and requires far less code than using standard Python lists.

2. Creating NumPy Arrays

First, you need to import the library:

```
import numpy as np
```

np.array()

The most basic way to create a NumPy array is by passing a Python list to the `np.array()` function.

```
# 1D Array (Vector)
```

```
a = np.array([1, 2, 3])
```

```
print(a)
```

```
# Output: [1 2 3]
```

```
# 2D Array (Matrix) - List of lists
```

```
b = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(b)
```

```
# Output:
```

```
# [[1 2 3]
```

```
# [4 5 6]]
```

```
# 3D Array (Tensor) - List of lists of lists
```

```
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

```
print(c)
```

Creating arrays with a specific data type:

By default, Python uses integers (`int64`), but you can specify a different data type.

```
# Float
```

```
np.array([1, 2, 3], dtype=float)
```

```
# Output: array([1., 2., 3.])
```

```
# Boolean (Any non-zero value is True, 0 is False)
```

```
np.array([1, 2, 0], dtype=bool)
```

```
# Output: array([ True,  True, False])
```

```
# Complex
```

```
np.array([1, 2, 3], dtype=complex)
# Output: array([1.+0.j, 2.+0.j, 3.+0.j])
```

np.arange()

Works exactly like Python's range() function but returns a NumPy array.

```
np.arange(1, 11)
# Output: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```
np.arange(1, 11, 2) # Adding a step of 2
# Output: array([1, 3, 5, 7, 9])
```

reshape()

Used to change the shape (dimensions) of an array. The product of rows and columns must equal the total number of items.

```
np.arange(1, 11).reshape(2, 5) # 2 rows, 5 columns
# Output:
# [[ 1  2  3  4  5]
#  [ 6  7  8  9 10]]
```

```
np.arange(1, 13).reshape(3, 4) # 3 rows, 4 columns
```

np.ones() and np.zeros()

Creates an array initialized with entirely 1s or 0s. Very useful in Neural Networks (e.g., initializing weights).

```
np.ones((3, 4)) # Pass shape as a tuple
np.zeros((3, 4))
```

np.random.random()

Creates an array of a given shape and populates it with random floats between 0.0 and 1.0.

```
np.random.random((3, 4))
```

np.linspace()

Generates linearly spaced numbers within a given range. It guarantees that the distance between consecutive points is exactly the same.

```
# Generate 10 equal distance points between -10 and 10  
np.linspace(-10, 10, 10)
```

np.identity()

Creates an identity matrix (a square matrix where diagonal values are 1 and the rest are 0).

```
np.identity(3) # 3x3 identity matrix
```

3. Array Attributes

To perform these attribute checks, let's assume we have 3 arrays:

- a1 (1D Vector)
- a2 (2D Matrix)
- a3 (3D Tensor)

1. ndim - Tells you the number of dimensions of the array.

```
a1.ndim # Output: 1  
a2.ndim # Output: 2  
a3.ndim # Output: 3
```

2. shape - Tells you the structure of the array (how many items in each dimension).

```
a1.shape # Output: (10,) - 10 items in 1 dimension  
a2.shape # Output: (3, 4) - 3 rows, 4 columns  
a3.shape # Output: (2, 2, 2) - Two 2x2 matrices
```

3. size - Total number of elements inside the array.

```
a2.size # Output: 12 (since it is a 3x4 matrix)
```

4. itemsize - Memory occupied by a single item in bytes.

```
a1.itemsize # Output: 8 (Because int64 takes 8 bytes)
```

5. dtype - Identifies the data type of the elements inside the array.

```
a1.dtype # Output: dtype('int64')
```

4. Changing Data Types (astype)

Used to optimize memory. If an array holds values representing age (e.g., 0-100), storing them as int64 wastes memory. Converting them to int32 reduces memory footprint dramatically, which is crucial when dealing with massive datasets.

```
# Check current memory footprint
print(a3.dtype) # int64
```

```
# Convert to smaller integer scale
a3 = a3.astype(np.int32)
print(a3.dtype) # int32
```

5. Array Operations

Scalar Operations

Performing mathematical operations on an array using a single number (a scalar). The operation is applied to *every single element* in the array.

```
a1 = np.arange(12).reshape(3,4)
```

```
a1 * 2 # Multiplies every element by 2
a1 + 5 # Adds 5 to every element
a1 ** 2 # Squares every element
```

```
# Relational scalar operations (returns boolean arrays)
a1 > 5
a1 == 15
```

Vector Operations

Performing operations between two NumPy arrays. The operation happens item-by-item (elements at corresponding index positions operate with each other). Note: arrays should be of identical shapes.

```
a1 = np.arange(12).reshape(3,4)
a2 = np.arange(12, 24).reshape(3,4)
```

```
a1 + a2 # Adds a1 to a2 item-wise  
a1 * a2  
a1 ** a2
```

6. Array Functions

NumPy comes packed with built-in mathematical functions, saving data scientists from writing everything from scratch.

Basic Mathematical Functions

```
a1 = np.random.random((3, 3))  
a1 = np.round(a1 * 100)
```

```
np.max(a1) # Finds max value in entire array  
np.min(a1) # Finds min value in entire array  
np.sum(a1) # Sum of all elements  
np.prod(a1) # Product of all elements
```

Finding Max/Min/Sum row-wise or column-wise (axis parameter)

- axis = 0: Operates column-wise
- axis = 1: Operates row-wise

```
np.max(a1, axis=0) # Returns max item from each column  
np.sum(a1, axis=1) # Returns sum of each row
```

Statistical Functions

```
np.mean(a1) # Mean of array  
np.median(a1) # Median  
np.std(a1) # Standard Deviation  
np.var(a1) # Variance
```

Dot Product

Matrix multiplication. To perform a dot product, the number of columns in the first matrix must match the number of rows in the second matrix (e.g., (3,4) dot (4,3) is valid, returning (3,3)).

```
a2 = np.arange(12).reshape(3, 4)  
a3 = np.arange(12, 24).reshape(4, 3)
```

```
np.dot(a2, a3)
```

Others

```
np.log(a1) # Calculates log of every item  
np.exp(a1) # Calculates exponent of every item  
np.round(a1) # Normal mathematical rounding  
np.floor(a1) # Rounds down (e.g., 6.9 -> 6.0)  
np.ceil(a1) # Rounds up (e.g., 6.1 -> 7.0)
```

7. Indexing and Slicing

Fetching single or multiple items from arrays.

1D Arrays

Works exactly like standard Python string/list indexing.

```
a1 = np.arange(10)  
a1[-1] # Fetch last item  
a1[2:5] # Slice from index 2 to 4
```

2D Arrays

Syntax: array[row_position, column_position]

```
a2 = np.arange(12).reshape(3, 4)  
# [[ 0, 1, 2, 3],  
# [ 4, 5, 6, 7],  
# [ 8, 9, 10, 11]]
```

```
# Fetch the number 6: It's in row 1, col 2  
a2[1, 2]
```

```
# Fetch the entire 1st row  
a2[0, :]
```

```
# Fetch the entire 3rd column (2, 6, 10)  
a2[:, 2]
```

```
# Fetch a sub-matrix containing [5, 6] and [9, 10]
```

```
a2[1:3, 1:3]
```

```
# Fetch alternate columns of alternating rows [0, 2] and [8, 10]
```

```
a2[:, ::2, ::2]
```

3D Arrays

Syntax: `array[2d_matrix_index, row_index, col_index]`

```
a3 = np.arange(27).reshape(3, 3, 3) # Contains 3 separate 3x3 matrices
```

```
# Fetch the middle 3x3 matrix
```

```
a3[1]
```

```
# Fetch the 1st row of the first 3x3 matrix
```

```
a3[0, 1, :]
```

```
# Fetch the middle column of the 2nd matrix
```

```
a3[1, :, 1]
```

8. Iterating Arrays

Looping over arrays.

```
# 1D Array
```

```
for i in a1:
```

```
    print(i) # Prints each item
```

```
# 2D Array
```

```
for i in a2:
```

```
    print(i) # Prints row by row
```

```
# 3D Array
```

```
for i in a3:
```

```
    print(i) # Prints 2D matrix by 2D matrix
```

```
# Iterating over every single item regardless of array dimensions using nditer()
```

```
for i in np.nditer(a3):
```

```
    print(i)
```


9. Advanced Reshaping

1. Transpose: Swaps rows to columns and columns to rows.

```
np.transpose(a2)
# OR shortcut syntax
a2.T
```

2. Ravel: Flattens any multi-dimensional array into a 1-Dimensional sequence.

```
a3.ravel()
```

10. Stacking

Stacking merges two or more NumPy arrays together. They must have identical shapes. Useful for merging data from multiple sources (e.g., some from DB, some from web scraping).

- **Horizontal Stacking (hstack):** Places arrays side by side.
- **Vertical Stacking (vstack):** Places arrays one on top of the other.

```
a4 = np.arange(12).reshape(3,4)
a5 = np.arange(12, 24).reshape(3,4)
```

```
np.hstack((a4, a5)) # Returns a 3x8 matrix
np.vstack((a4, a5)) # Returns a 6x4 matrix
```

11. Splitting

The reverse of stacking. Breaks an array into multiple equal parts.

- **Horizontal Split (hsplit):** Slices columns vertically.
- **Vertical Split (vsplit):** Slices rows horizontally.
(Note: Visual naming can feel counterintuitive at first! Think of hsplit as cutting through the horizontal x-axis and vsplit cutting through the vertical y-axis).

```
# Split horizontally into 2 equal parts
np.hsplit(a4, 2)
```

```
# Split vertically into 3 equal parts
np.vsplit(a4, 3)
```

Note: You must specify a number that evenly divides the array; otherwise, NumPy throws an

error.

Summary of Lecture:

The lecture covered the core basics of NumPy: Array creation, shaping, mathematical operations, slicing/indexing logic (especially 2D and 3D), and structural manipulation tools like stacking, splitting, ravel, and transpose. These form the complete bedrock needed before diving into Advanced NumPy and Data Manipulation via Pandas.